

Using the Duke Compute Cluster

SSH, filesystems, Slurm, GPUs, and the batch-job lifecycle — with runnable examples.

Companion: *Guide 2 – PINN Cluster Scripts* (this project's `slurm/` tree) | Group: `fisherlab` | Drafted 2026-08-02

VERIFY BEFORE YOU RELY ON IT

Cluster configuration drifts — partition names, GPU inventory, quotas and time limits change without notice. Everything in this guide that is *site-specific* (host names, partitions, paths, quotas) is marked with a **check command** you can run to confirm the current truth. The authoritative source is Duke OIT's DCC documentation and `oit-rc-help@duke.edu`; this guide is a working companion, not a replacement.

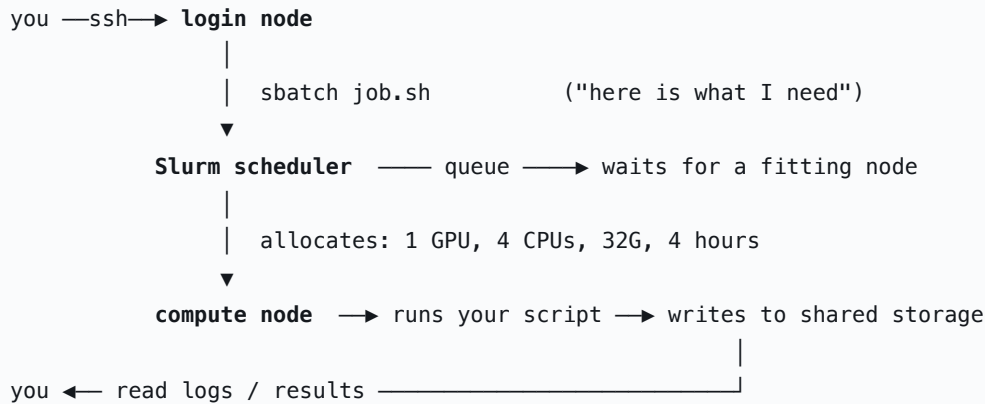
- | | |
|---------------------------------------|---------------------------------|
| 1. The mental model: login vs compute | 8. Interactive sessions |
| 2. Getting on: SSH and OnDemand | 9. Job arrays |
| 3. Where your files live | 10. Job dependencies |
| 4. Partitions, accounts, and GPUs | 11. Right-sizing your requests |
| 5. Software: modules and conda | 12. Moving data in and out |
| 6. Anatomy of an sbatch script | 13. Troubleshooting table |
| 7. The five commands you actually use | 14. Etiquette and the checklist |

1. The mental model: login vs compute

A cluster is not one computer. It is three distinct things that beginners routinely conflate, and almost every "why did my job die" question traces back to confusing them.

Layer	What it is	What you may do there
Login node	The shared machine you land on after <code>ssh</code> . A handful of cores shared by <i>everyone</i> logged in right now.	Edit files, <code>git</code> , submit jobs, inspect small outputs, <code>tail</code> logs. Nothing that runs longer than ~a minute or uses real memory.
Scheduler (Slurm)	A queue and a resource accountant. You describe a job; it finds a node that fits and runs it there.	Interact only through commands: <code>sbatch</code> , <code>squeue</code> , <code>scancel</code> , <code>sacct</code> , <code>srun</code> .
Compute nodes	The actual workhorses — many cores, lots of RAM, GPUs on some. You never SSH to them directly; Slurm places you there.	Everything expensive: training, long analyses, anything touching a GPU.

The single rule that follows from this: **never run your training script directly on the login node**. It will either be killed by the node's resource watchdog, or — worse — it will not be killed and will degrade the login node for every other person on the cluster. If you catch yourself typing `python run_walk_forward.py` right after `ssh`, stop.



Notice the last arrow: storage is *shared*. The compute node sees the same `/hpc/group/...` directory as the login node, which is why you do not need to copy anything to the compute node. Write your results to shared storage and they are simply there when the job finishes.

2. Getting on: SSH and OnDemand

2.1 SSH from a terminal

```
# from your laptop - replace ds555 with your NetID
ssh ds555@dcc-login.oit.duke.edu
```

You will be prompted for your Duke password and then for multi-factor authentication. Off the Duke network you generally need the Duke VPN running first.

Typing that host and doing MFA a dozen times a day gets old fast. Two fixes, both on your laptop:

An SSH config alias

Put this in `~/.ssh/config` on your laptop:

```
Host dcc
  HostName dcc-login.oit.duke.edu
  User ds555
  ServerAliveInterval 60
  ControlMaster auto
  ControlPath ~/.ssh/cm-%r@%h:%p
  ControlPersist 8h
```

Now `ssh dcc` is enough. The three `Control*` lines are the part worth understanding: the first connection opens a master channel that stays alive for 8 hours, and every subsequent `ssh dcc`, `scp`, or `rsync` rides that existing channel — no password, no MFA. `ServerAliveInterval` keeps the channel from being dropped by an idle timeout.

An SSH key

```
# laptop – generate once, if you don't already have a key
ssh-keygen -t ed25519 -C "ds555@duke.edu"

# copy the public half to the cluster
ssh-copy-id dcc
```

Duke may still require MFA on top of the key depending on current policy; the key at minimum removes the password step.

2.2 OnDemand (the browser route)

DCC provides an Open OnDemand portal — typically at `dcc-ondemand.oit.duke.edu` — which gives you, in a browser: a file manager, a terminal, and "interactive apps" such as JupyterLab or an RStudio session running *on a compute node*. Two things it is genuinely good for:

- **Looking at figures.** Your job writes a PNG into `reports/`; you click it in the file manager instead of `scp`-ing it down.
- **Exploratory notebook work on a GPU.** Launching a Jupyter session through OnDemand submits a real Slurm job, so the notebook kernel has the GPU and memory you asked for. This is the correct way to prototype interactively — much better than fighting `srun` plus port forwarding.

It is not good for launching production runs. Those belong in `sbatch` scripts, in git, so they are reproducible six months later.

3. Where your files live

DCC exposes several filesystems with very different sizes, speeds, and persistence guarantees. Putting the wrong thing in the wrong place is the second-most-common source of pain after the login-node mistake.

Path	Typical size	Persistence	Use it for
<code>/hpc/home/<netid></code>	small (a few GB)	backed up	Dotfiles, SSH keys, tiny scripts. Not code repos, not data, not conda envs — you will hit the quota and jobs will fail in confusing ways.
<code>/hpc/group/<lab></code> e.g. <code>/hpc/group/fisherlab</code>	large (TB-scale, purchased)	persistent	The default home for real work. Code, conda environments, datasets, run outputs, checkpoints. This project lives at <code>/hpc/group/fisherlab/ds555/pinn_code</code> .
<code>/work</code> or scratch	very large	auto-purged	High-throughput temporary I/O — intermediate files a job writes then re-reads. Assume anything untouched for a few weeks is deleted. Never the only copy of a result.
<code>/tmp</code> (node-local)	modest	gone at job end	Per-job scratch that never needs to leave the node. Fastest option, since it is a local disk rather than the network filesystem.

CHECK COMMAND

Confirm your actual quotas and what exists — do not trust the table above blindly:

```
quota -s          # home directory usage
df -h /hpc/group/fisherlab    # group space free/used
du -sh /hpc/group/fisherlab/ds555/*  # what is eating your share
ls -ld /work /scratch 2>/dev/null    # which scratch paths exist here
```

3.1 Why conda environments must not live in home

A PyTorch-with-CUDA environment is several gigabytes. Home quota is typically smaller than that. So the miniconda installation for this project deliberately sits in group space:

```
/hpc/group/fisherlab/ds555/miniconda3/
```

which is exactly the path every job script in `slurm/` sources. If you install miniconda into `~` by accepting the installer default, you will fill your home quota, and the symptom will not be "disk full" — it will be a torch import failing, or a job dying at epoch 3000 when it tries to write a checkpoint. Install to group space.

4. Partitions, accounts, and GPUs

A **partition** is a named pool of nodes with its own rules: which hardware, how long jobs may run, who may submit. You pick one with `--partition`.

Partition	Hardware	Character
<code>common</code>	CPU only	The general-purpose CPU pool, open to all Duke users. Where analysis and aggregation jobs belong.
<code>gpu-common</code>	GPU	The general GPU pool. Every job script in this project uses this one. Expect to queue during busy periods.
<code><lab></code> / <code><lab>--gpu</code>	varies	Nodes your lab purchased. Highest priority for lab members, but only if the lab owns nodes.
<code>scavenger</code> , <code>scavenger-gpu</code>	varies	Idle capacity from other groups' nodes. Often near-zero wait — but your job is preempted (killed) without warning when the owner needs the node. Only for work that checkpoints, or short work you don't mind losing.

CHECK COMMAND

```
sinfo -s          # all partitions, node counts, states
sinfo -p gpu-common -o "%20N %10c %10m %25G %10T"  # nodes / cores / mem / GRES / state
scontrol show partition gpu-common          # limits, incl. MaxTime and DefaultTime
```

4.1 Requesting a GPU

GPUs are a "generic resource" (GRES) in Slurm. Asking for one takes an explicit flag — **a job without it gets zero GPUs even on a GPU partition**, and your code will silently fall back to CPU and run perhaps fifty times slower.

```
#SBATCH --partition=gpu-common
#SBATCH --gres=gpu:2080:1      # 1 GPU, specifically an RTX 2080
```

The middle field is the GPU *type* and it is optional but consequential:

`--gres=gpu:1`

Any available GPU. Shortest queue wait, but you might land on anything from an old 2080 to an A100. Fine when your job fits comfortably in the smallest card.

`--gres=gpu:2080:1`

Specifically an RTX 2080 (11 GB). What this project requests. Deliberate: these networks are small, a 2080 is plenty, and asking for the modest card means a much shorter queue than competing for A100s. **Requesting a bigger GPU than you need makes your job wait longer for no speedup.**

`--gres=gpu:a100:1`

Specifically an A100 (40–80 GB). Justified only by genuine memory pressure or a model large enough to exploit it.

To see which type names are actually valid right now:

```
sinfo -p gpu-common -o "%N %G" | sort -u
```

4.2 Accounts

Some clusters require `#SBATCH --account=<something>` to charge the job against an allocation. At Duke this is generally handled by your group membership, and none of this project's scripts set it — they work as-is. If you ever see *"Invalid account or account/partition combination specified,"* that is the flag you are missing. Find your valid accounts with:

```
sacctmgr show associations user=$USER format=Account,Partition,QOS
```

5. Software: modules and conda

5.1 Environment modules

Cluster-provided software is exposed through `module`, which manipulates `PATH` and friends so that several conflicting versions of a tool can coexist on one machine.

```
module avail          # everything installed (long)
module avail cuda     # filter by name
module load CUDA/12.1 # put it on PATH for this shell
module list           # what's loaded right now
module unload CUDA/12.1
module purge          # clean slate – good first line in a job script
```

This project's scripts do *not* load a CUDA module, because the PyTorch installed by pip ships its own bundled CUDA runtime (the `+cu121` wheel in `environment_server.yml`). Only the NVIDIA *driver* needs to come from the node, and that is always present. Loading a conflicting CUDA module on top is a common way to break a working torch install.

5.2 Conda: one-time setup

Do this once, on a login node — installing packages is I/O-bound, not compute-bound, so it is an acceptable login-node activity.

```
# 1. Install miniconda into GROUP space, not home (see §3.1)
cd /hpc/group/fisherlab/ds555
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p /hpc/group/fisherlab/ds555/miniconda3

# 2. Make conda usable in THIS shell (note: source, not conda init)
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh

# 3. Build the project environment from the LINUX spec file
cd /hpc/group/fisherlab/ds555/pinn_code
conda env create -f environment_server.yml
conda activate pinn_env
```

USE ENVIRONMENT_SERVER.YML, NEVER ENVIRONMENT.YML

`environment.yml` is the macOS spec — it pins `pyobjc`, `appnope` and other Apple-only packages that cannot resolve on Linux. `environment_server.yml` is the cluster one, and it installs the CUDA-enabled PyTorch wheel. Match the wheel's CUDA version to the node driver: check with `nvidia-smi` inside a GPU allocation and adjust the `--index-url` (`cu118` / `cu121` / `cu124`) if needed.

5.3 Why job scripts say `source ... conda.sh`

Every script in `slurm/` begins with the same two lines:

```
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env
```

The reason is that a batch job starts a **non-interactive shell**. Non-interactive shells do not read `~/.bashrc`, which is where `conda init` writes its setup block. So the `conda` shell function simply does not exist inside a job — and `conda activate` fails with *"Your shell has not been properly configured."* Sourcing `conda.sh` explicitly defines the function and makes the script self-contained regardless of the submitting user's dotfiles. Do this in every job script; it is not optional boilerplate.

5.4 Verifying the GPU actually arrived

Put this near the top of any new GPU job. It costs two seconds and saves you from discovering at hour three that you trained on CPU:

```
python -c "import torch; print('CUDA:', torch.cuda.is_available()); print('Device:',
torch.cuda.get_device_name(0))"
```

`slurm/run_smoke_test.sh` and `slurm/run_w1_smoke.sh` both do exactly this — that is what makes them smoke tests rather than just short runs.

6. Anatomy of an sbatch script

A batch script is an ordinary shell script with a header of `#SBATCH` comment lines. The shell ignores them; Slurm reads them at submission to size the job. Here is a complete, annotated example — deliberately close to this project's real ones.

```
#!/bin/bash
#----- resource request -----
#SBATCH --job-name=pinn_demo           # label shown by squeue
#SBATCH --partition=gpu-common         # which node pool
#SBATCH --gres=gpu:2080:1             # 1 RTX 2080
#SBATCH --cpus-per-task=4             # CPU cores (dataloader workers, BLAS)
#SBATCH --mem=32G                     # HOST RAM — separate from GPU memory
#SBATCH --time=4:00:00                # HH:MM:SS wall clock; job is KILLED at the limit
#SBATCH --output=slurm/logs/demo_%j.out # stdout; %j = job ID
#SBATCH --error=slurm/logs/demo_%j.err  # stderr
# Optional but useful:
#SBATCH --mail-type=END,FAIL
#SBATCH --mail-user=ds555@duke.edu

#----- environment -----
cd /hpc/group/fisherlab/ds555/pinn_code
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env

#----- provenance header -----
# Print WHAT ran WHERE. Six months from now this log is your only
# record of which node and which GPU produced these numbers.
echo "Job ID:  $SLURM_JOB_ID"
echo "Node:    $SLURMD_NODENAME"
echo "GPU:     $(nvidia-smi --query-gpu=name --format=csv,noheader)"
echo "Commit:  $(git rev-parse --short HEAD)"
echo "Date:    $(date)"

#----- the actual work -----
python run_walk_forward.py \
    --arch modified --mode hybrid --rwf_mu 0.75 \
    --epochs 15000 --folds Nov2020 \
    --output_dir runs/v2/demo

echo "Done: $(date)"
```

6.1 The directives that matter, and how to get them wrong

Directive	What it means	Failure mode if wrong
<code>--time</code>	Wall-clock ceiling. The job is hard-killed the instant it is exceeded.	Too low: job dies at 99% with no result and a truncated log. Too high: the scheduler can't slot you into short gaps, so you queue much longer. Aim for $\sim 1.5\times$ your measured runtime.
<code>--mem</code>	Host RAM. Nothing to do with GPU VRAM.	Exceeding it triggers an OOM kill — often reported cryptically as exit code 137 or "Killed", with no Python traceback.
<code>--gres</code>	GPU count and optionally type.	Omitted → <code>torch.cuda.is_available()</code> is <code>False</code> , training silently proceeds on CPU at a crawl.
<code>--cpus-per-task</code>	Cores for the one task.	Too few starves dataloader workers; too many just extends your queue wait. For this project 4 is right — the work is GPU-bound.
<code>--output/--error</code>	Log destinations. <code>%j</code> = job ID, <code>%A</code> = array parent ID, <code>%a</code> = array task index.	The directory must already exist. Slurm will not create <code>slurm/logs/</code> for you — the job fails instantly and, cruelly, leaves no log explaining why.

THE MKDIR TRAP

`mkdir -p slurm/logs` must happen *before* submission, not inside the script — by the time the script body runs, Slurm has already tried and failed to open the log file. This is exactly why `slurm/submit_all.sh` and `slurm/submit_w1.sh` both call `mkdir -p slurm/logs` before their first `sbatch`.

6.2 Useful environment variables inside a job

Variable	Value
<code>\$SLURM_JOB_ID</code>	Unique job ID. Log it.
<code>\$SLURM_ARRAY_TASK_ID</code>	This task's index within an array — the mechanism behind every config lookup in this project.
<code>\$SLURM_ARRAY_JOB_ID</code>	The array's parent ID, shared by all its tasks.
<code>\$SLURMD_NODENAME</code>	Which physical node you got.
<code>\$SLURM_CPUS_PER_TASK</code>	Cores actually granted — pass it to your dataloader instead of hardcoding.
<code>\$SLURM_SUBMIT_DIR</code>	The directory you ran <code>sbatch</code> from.

7. The five commands you actually use

sbatch — submit

```
sbatch slurm/run_smoke_test.sh      # → "Submitted batch job 1234567"
sbatch --parsable slurm/run_smoke_test.sh  # → "1234567" (bare ID, for scripting)
sbatch --time=8:00:00 job.sh        # CLI flags OVERRIDE #SBATCH lines
sbatch --array=0-12 slurm/run_benchmark.sh  # submit as a 13-task array
```

The override rule is worth internalizing: anything on the command line beats the corresponding `#SBATCH` line in the file. That is how you extend a time limit for one run without editing (and having to un-edit) a script that is under version control.

squeue — what is running or waiting

```
squeue -u $USER          # your jobs
squeue -u $USER -t PENDING # only the queued ones
squeue -j 1234567        # one specific job
squeue -u $USER --start   # estimated start times (a guess, not a promise)
watch -n 30 "squeue -u $USER" # auto-refresh every 30s (q to quit)
```

The `ST` column is the state, and the `NODELIST(REASON)` column tells you why a pending job is pending:

Code / reason	Meaning
R	Running.
PD	Pending — see the reason in the last column.
CG	Completing — finished, cleaning up.
Resources	Normal. Nothing matching your request is free yet. Wait.
Priority	Normal. Higher-priority jobs are ahead of you.
Dependency	Waiting on a job you told it to wait for (see §10). Normal in this project's smoke→array pattern.
DependencyNeverSatisfied	Not normal. The prerequisite job failed, so this one will wait forever. Cancel it and fix the prerequisite.
QOSMaxJobsPerUserLimit	You have hit a concurrency cap. Later tasks start as earlier ones finish; nothing to fix.
ReqNodeNotAvail	You asked for something that does not exist here — often a GPU type name that is wrong, or a <code>--time</code> above the partition maximum.

scancel — stop

```
scancel 1234567          # one job (or a whole array, by parent ID)
scancel 1234567_3        # one task of an array
scancel -u $USER          # EVERYTHING you have queued or running – be sure
scancel -u $USER -t PENDING # drop the queue, let running jobs finish
scancel --name=pinn_w1_balancer # by job name
```

sacct — history and post-mortems

`squeue` only shows live jobs. Once a job ends it vanishes from `squeue` entirely, and `sacct` is how you find out what happened.

```
sacct -j 1234567 --format=JobID,JobName%22,State,ExitCode,Elapsed,MaxRSS,ReqTRES%40
sacct -u $USER --starttime=2026-08-01 --format=JobID,JobName%20,State,Elapsed
sacct -u $USER --state=FAILED,TIMEOUT --starttime=now-7days # the week's failures
```

Read the `State` column first:

State	What it tells you
COMPLETED	Exit code 0. Note this only means the <i>script</i> succeeded — a Python script that caught an exception and returned 0 still shows COMPLETED. Check your logs.
FAILED	Non-zero exit. Read the <code>.err</code> file.
TIMEOUT	Hit <code>--time</code> . Raise the limit, or checkpoint and resume.
OUT_OF_MEMORY	Exceeded <code>--mem</code> . Raise it, or shrink batch/grid size.
CANCELLED	You (or an admin, or preemption on <code>scavenger</code>) killed it.
NODE_FAIL	Hardware problem, not your fault. Resubmit.

seff — the efficiency report

Run this on every finished job for a while. It is the fastest way to learn what your jobs actually need.

```
seff 1234567
```

```
Job ID: 1234567
State: COMPLETED (exit code 0)
Cores: 4
CPU Utilized: 00:41:12
CPU Efficiency: 8.4% of 08:12:00 core-walltime
Job Wall-clock time: 02:03:00
Memory Utilized: 3.21 GB
Memory Efficiency: 10.03% of 32.00 GB
```

Read that output as a diagnosis, not a grade:

- **Memory Efficiency 10%** — you asked for 32 G and used 3.2 G. Drop to `--mem=8G`. Smaller requests are scheduled sooner, so this directly shortens your queue wait.
- **CPU Efficiency 8%** — expected and fine for GPU training. The CPU is idle because the GPU is doing the work. `seff` does not measure GPU utilization, so low CPU efficiency here is not a problem to fix.
- **Wall-clock 2:03 against a 4-hour limit** — reasonable headroom. Below $\sim 1.2\times$ you risk TIMEOUT; above $\sim 3\times$ you are queueing longer than necessary.

8. Interactive sessions

Batch jobs are right for anything you will run more than once. But for debugging — when you want to run a script, watch it crash, edit, and rerun — you want a shell on a compute node.

```
# Ask for a GPU node and drop into a shell on it
srun --partition=gpu-common --gres=gpu:2080:1 \
    --mem=16G --cpus-per-task=4 --time=1:00:00 --pty bash

# You are now ON a compute node. Set up and work normally:
cd /hpc/group/fisherlab/ds555/pinn_code
source /hpc/group/fisherlab/ds555/miniconda3/etc/profile.d/conda.sh
conda activate pinn_env
python run_walk_forward.py --folds Nov2020 --epochs 200 --output_dir runs/v2/scratch

exit # releases the allocation - do NOT just close your laptop
```

Three cautions:

- **The allocation dies with your SSH connection.** Wifi drops, laptop sleeps, session gone. Run `tmux` on the login node *before* `srun`, and you can detach (`Ctrl-b d`) and reattach later with `tmux attach`.
- **You are holding a GPU the entire time**, including while you stare at code. Ask for one hour, not eight, and `exit` when you are done thinking.
- **Interactive work is not reproducible.** The moment something works, move it into a script under `slurm/` and commit it. `slurm/run_w0_audits.sh` documents its own `srun` equivalent in a comment block for exactly this reason — the batch script is the record; the interactive form is the scratchpad.

9. Job arrays

An array runs the same script N times with a different index each time. It is the correct tool whenever you have a grid of configurations — which, in this project, is essentially always.

```
sbatch --array=0-12 slurm/run_benchmark.sh # 13 tasks: 0,1,...,12
sbatch --array=0,3,7 job.sh               # only those three
sbatch --array=0-12%4 job.sh              # at most 4 concurrent (be a good neighbour)
sbatch --array=0-5 --dependency=afterok:999 job.sh
```

Inside the script, `$SLURM_ARRAY_TASK_ID` holds this task's index. The pattern used throughout this project is a parallel-arrays lookup — index into bash arrays to recover the configuration:

```
# One entry per task, positions aligned across all arrays
ARCHS=(standard standard modified modified)
MODES=(physics hybrid physics hybrid)
MUS=( 1.0 1.0 0.75 0.75)

ARCH=${ARCHS[$SLURM_ARRAY_TASK_ID]}
MODE=${MODES[$SLURM_ARRAY_TASK_ID]}
MU=${MUS[$SLURM_ARRAY_TASK_ID]}

python run_walk_forward.py --arch $ARCH --mode $MODE --rwf_mu $MU
```

This is readable, has no dependencies, and keeps the whole grid visible in one place. Its one hazard is silent misalignment: if you append to `ARCHS` and forget `MUS`, the missing element expands to an empty string, and your job runs with `--rwf_mu` followed by nothing — either a confusing argparse error or, worse, a run with the wrong meaning. Whenever you extend a grid, count every array.

Log files should use `%A_%a` so each task gets its own:

```
#SBATCH --output=slurm/logs/slurm_%A_%a.out # → slurm_1234567_0.out, _1.out, ...
```

9.1 Rerunning just the tasks that failed

```
# Which tasks of array 1234567 failed?
sacct -j 1234567 --format=JobID,State | grep -E "FAILED|TIMEOUT"

# Resubmit only those indices – no need to rerun the successes
sbatch --array=3,7 slurm/run_benchmark.sh
```

This is a major practical advantage of the index-lookup pattern: task index 7 always means the same configuration, so re-running index 7 reproduces exactly the cell that failed.

10. Job dependencies

Dependencies let you chain jobs so that stage 2 only starts if stage 1 succeeded. This project uses them to gate expensive arrays behind cheap smoke tests.

```
# --parsable makes sbatch print ONLY the job ID, so it can be captured
SMOKE_ID=$(sbatch --parsable slurm/run_smoke_test.sh)

# The array is submitted immediately but stays PENDING until smoke succeeds
sbatch --array=0-12 --dependency=afterok:$SMOKE_ID slurm/run_benchmark.sh
```

Dependency type	Starts when the prerequisite...
<code>afterok:ID</code>	...finishes with exit code 0. The one you almost always want.
<code>afterany:ID</code>	...finishes, success or not. For cleanup or "collect whatever we got" jobs.
<code>afternotok:ID</code>	...fails. For automatic notification or fallback.
<code>after:ID</code>	...merely starts. Rarely what you mean.
<code>singleton</code>	...all your prior jobs with the same <code>--job-name</code> are done. Serializes a named pipeline.

The payoff: a 13-task array on a 4-hour limit is up to 52 GPU-hours. A five-minute smoke test in front of it catches the typo, the missing flag, and the broken import before any of that is committed. This is why `submit_all.sh`, `submit_stage2.sh`, and `submit_w1.sh` all follow the same smoke-gates-array shape.

DEPENDENCYNEVERSATISFIED

If the smoke test fails, the dependent array does not fail — it sits `PD` forever with reason `DependencyNeverSatisfied`, quietly occupying a slot in your queue. Slurm may eventually purge it, but do not count on that. Check with `squeue -u $USER` after any failed gate and `scancel` the orphan.

11. Right-sizing your requests

There is a real tension here. Bigger requests are safer (no TIMEOUT, no OOM) but queue longer, because the scheduler must find a larger contiguous hole. Smaller requests start sooner but risk being killed. The resolution is to measure rather than guess.

1. **Smoke run** — one fold, few epochs, generous limits (`--mem=32G --time=1:00:00`)
2. **seff it** — read actual MaxRSS and Elapsed
3. **Extrapolate** — 9 folds $\times$ 15000 epochs is $\sim 90\times$ a 1-fold 1500-epoch run
4. **Set limits** — memory $\approx 1.5\times$ observed peak; time $\approx 1.5\times$ extrapolated
5. **Submit the array**

Two asymmetries to keep in mind while doing this. Memory overshoot is cheap in wall-clock terms and catastrophic when wrong, so be generous — 32 G costs little on a node with 256 G. Time overshoot is genuinely expensive in queue position, so be closer to the bone there, and prefer checkpointing over enormous limits for very long jobs.

12. Moving data in and out

```
# Laptop → cluster (single file)
scp data/TSLA_2020_Split_Adjusted.csv dcc:/hpc/group/fisherlab/ds555/pinn_code/data/

# Cluster → laptop (results directory), resumable and incremental
rsync -avz --progress dcc:/hpc/group/fisherlab/ds555/pinn_code/reports/ ./reports/

# Sync code up while skipping everything that shouldn't travel
rsync -avz --exclude '.git' --exclude '__pycache__' --exclude 'runs/' \
    ./ dcc:/hpc/group/fisherlab/ds555/pinn_code/
```

Prefer `rsync` over `scp` for anything you will transfer more than once: it copies only what changed, resumes after an interruption, and `-z` compresses in flight. Note that with the `ControlMaster` config from §2.1, none of these re-prompt for MFA.

That said, for *code* the better path is git — push from your laptop, pull on the cluster. It gives you the commit hash in your job log, which is what makes a run reproducible. Reserve `rsync` for data and results, which do not belong in git anyway.

13. Troubleshooting table

Symptom	Likely cause	Fix
Job vanishes instantly, no log file at all	The log directory does not exist	<code>mkdir -p slurm/logs</code> before submitting
<code>conda: command not found</code>	Non-interactive shell never read <code>.bashrc</code>	<code>source ../etc/profile.d/conda.sh</code> in the script (§5.3)
<code>CUDA: False</code> in the log	<code>--gres</code> missing, or wrong partition	Add <code>--gres=gpu:2080:1</code> and <code>--partition=gpu-common</code>
<code>CUDA out of memory</code>	GPU VRAM exhausted — <code>--mem</code> is irrelevant here	Smaller batch/collocation grid, or request a larger GPU type
Exit 137, "Killed", no traceback	Host RAM OOM	Raise <code>--mem</code> ; confirm with <code>sacct --format=MaxRSS</code>
State <code>TIMEOUT</code>	<code>--time</code> too low	Raise it; <code>seff</code> the partial run to extrapolate
Pending forever, <code>ReqNodeNotAvail</code>	Impossible request — bad GPU type name, or time above partition max	<code>sinfo -p gpu-common -o "%N %G"; scontrol show partition gpu-common</code>
Pending, <code>DependencyNeverSatisfied</code>	The gating job failed	Read the gate's log, <code>scancel</code> the orphan, fix, resubmit
<code>Permission denied</code> writing output	Writing outside your group space, or quota exhausted	<code>df -h /hpc/group/fisherlab</code> ; check the path in <code>--output_dir</code>
Runs on login node but fails in batch	Relative paths — the job's CWD is not what you assume	<code>cd</code> to an absolute project root as the script's first line
Array task N wrong config	Misaligned parallel bash arrays	Count every array; <code>echo</code> the resolved config before running (§9)
Job killed mid-run, State <code>PREEMPTED</code>	Running on <code>scavenger</code> ; owner reclaimed the node	Expected there. Move to <code>gpu-common</code> , or checkpoint and auto-resume

13.1 A general debugging order

1. `sacct -j <id> --format=JobID,State,ExitCode,Elapsed,MaxRSS` — did Slurm kill it, or did the code fail?
2. If Slurm killed it, the State names the resource. Raise that one.
3. If the code failed, read the `.err` file — the traceback is there.
4. If the `.err` is empty and the `.out` stops mid-stream, suspect OOM.
5. Reproduce interactively with `srunk --pty bash` on the smallest input that still fails.

14. Etiquette and the checklist

Shared infrastructure runs on norms as much as on rules.

- **Nothing heavy on login nodes.** No training, no big compiles, no `python` sessions loading a dataset. Editing, git, and `tail` are fine.
- **Ask for what you need.** Habitually requesting 8 GPUs and 256 G "to be safe" blocks other people and lengthens your own queue.
- **Throttle wide arrays** with `%N` — `--array=0-99%10` is a courtesy that costs you almost nothing.
- **Cancel what you don't need.** Especially interactive allocations, which sit idle holding a GPU while you think.
- **Clean up scratch.** Delete failed-run directories; group quota is shared with your labmates.

- **Prefer `scavenger` for preemptible work.** If your job checkpoints and resumes, using idle capacity is free throughput for you and costs the owner nothing.

PRE-SUBMISSION CHECKLIST

1. `mkdir -p slurm/logs` — the directory exists
2. Script `cd` s to an absolute project root
3. Conda is sourced explicitly, then activated
4. `--gres` present if the job needs a GPU
5. `--time` and `--mem` informed by a prior `seff` , not by guesswork
6. Output paths point where you intend (for this project: check `RUN_ROOT` — see Guide 2)
7. A short smoke test has passed, ideally as an `afterok` gate
8. Job name is descriptive enough to recognize in `squeue` next week

Where to go next. Guide 2 applies all of this to the PINN project's own `slurm/` tree: what each script does, the `RUN_ROOT` versioning that keeps v1 and v2 results separate, and complete end-to-end runthroughs. For anything site-specific that this guide could not pin down, the Duke OIT DCC documentation and `oit-rc-help@duke.edu` are authoritative.